МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра систем автоматизованого проектування

ЗВІТ

до лабораторної роботи № 6
з дисципліни «Технології веброзробки та дизайну»

на тему: «Верстка UI-компонентів та адаптивність за допомогою Tailwind CSS та ШІ-асистентів»

Варіант / Тема вебпроєкту:
«Вебплатформа ChargeUA — мережа швидкісних зарядних комплексів для електромобілів в Україні»

Виконав:
студент групи ПП-24
Лемішка Андрій Андрійович

Перевірила:
к.т.н., доц. кафедри САПР
Юрчак Ірина Юріївна

Оцінка: _____
Підпис викладача: _____

Львів — 2026

1. Мета та завдання лабораторної роботи

Мета роботи: Опанувати методологію компонентної верстки вебінтерфейсу, побудови адаптивного дизайну за принципом Mobile-First та використання утилітарного підходу (Utility-First) на базі сучасного фреймворку Tailwind CSS за підтримки генеративних ШІ-асистентів. Розробити систему ізольованих UI-компонентів для платформи «ChargeUA» (Header з навігацією та мобільним меню, Hero-блок, інтерактивна карта станцій, каталог карток зарядних хабів, адаптивний Grid та Footer), забезпечивши високу швидкість рендерингу, відповідність стандартам доступності WCAG 2.2 AA та сумісність з дизайн-системою ЛР №4.

Основні завдання роботи:

1. Налаштувати робоче середовище Visual Studio Code та встановити обов'язкові розширення (Tailwind CSS IntelliSense, Prettier Plugin for Tailwind CSS, Inline Fold).
2. Опанувати концепцію Utility-First CSS та порівняти класичний підхід CSS із Tailwind на прикладі UI-компонентів.
3. Підключити Tailwind CSS до проєкту за допомогою офіційного CDN-скрипту та узгодити його з таблицею стилів style.css.
4. Ознайомитися з інтерактивною онлайн-пісочницею Tailwind Play та провести тестування атомарних стилів і станів hover.
5. Спроєктувати та згенерувати за допомогою ШІ перший UI-компонент кнопки (Primary CTA Button) з розміром мішені min 44px.
6. Згенерувати адаптивний Header із логотипом ліворуч, горизонтальною навігацією на десктопах (md:flex) та мобільним меню-гамбургером (md:hidden).
7. Засвоїти та реалізувати на практиці принцип Mobile-First (базові класи для смартфонів, прогресивне розширення через sm:, md:, lg:).
8. Створити компонент картки зарядної станції (Card) з фотографією, технічними параметрами портів, статусами та кнопкою дій.
9. Побудувати адаптивну сітку карток за допомогою Tailwind CSS Grid (1 колонка mobile -> sm:grid-cols-2 -> lg:grid-cols-3, gap-6).
10. Зверстати семантичний 4-колонковий підвал сайту (Footer) з описом бренду, навігацією, послугами та контактами підтримки.
11. Провести тестування адаптивності та інтерактивності компонентів у Tailwind Play.
12. Здійснити інженерний аналіз згенерованого ШІ-коду (виявити правильні рішення, сумнівні конструкції, усунути зайві arbitrary values).
13. Інтегрувати створені компоненти у вебпроєкт ChargeUA зі збереженням семантичної структури стандарту HTML5.
14. Перенести дизайн-систему ChargeUA (кольори, шрифти, радіуси, тіні) у Tailwind та розподілити правила між класами й custom CSS.
15. Налаштувати брейкпоїнти адаптивності для сітки, розміру шрифтів, відступів та кнопок.
16. Провести тестування сумісності в браузері через Chrome DevTools Device Toolbar (320px, 768px, 1280px).
17. Виконати рафінування коду: перевірити семантику, доступність (WCAG 2.2 AA), alt-підписи та плавні переходи (transition).
18. Підготувати проєкт до хмарного розгортання на сервісі Netlify.
19. Провести валідацію за стандартами W3C Markup Validation Service та W3C CSS Validator.

2. Покроковий хід виконання роботи

Етап 1. Підготовка проєкту та робочого середовища

Роботу продовжено на базі вебпроєкту Лабораторної роботи №5. Перед внесенням змін створено повну резервну копію кодової бази ЛР №5. Структура проєкту містить: index.html, папку css/ зі style.css, assets/ (images/, icons/, fonts/, data/) та js/. У редакторі Visual Studio Code встановлено та

активовано спеціалізовані розширення:
• Tailwind CSS IntelliSense — надає інтелектуальне автодоповнення утилітарних класів, підсвічування та відображення згенерованого CSS-коду;
• Prettier Plugin for Tailwind CSS — автоматично сортує та впорядковує класи Tailwind за рекомендованим офіційним порядком;
• Inline Fold — візуально згортає довгі переліки утилітарних класів всередині рядка для покращення читабельності HTML-розмітки.

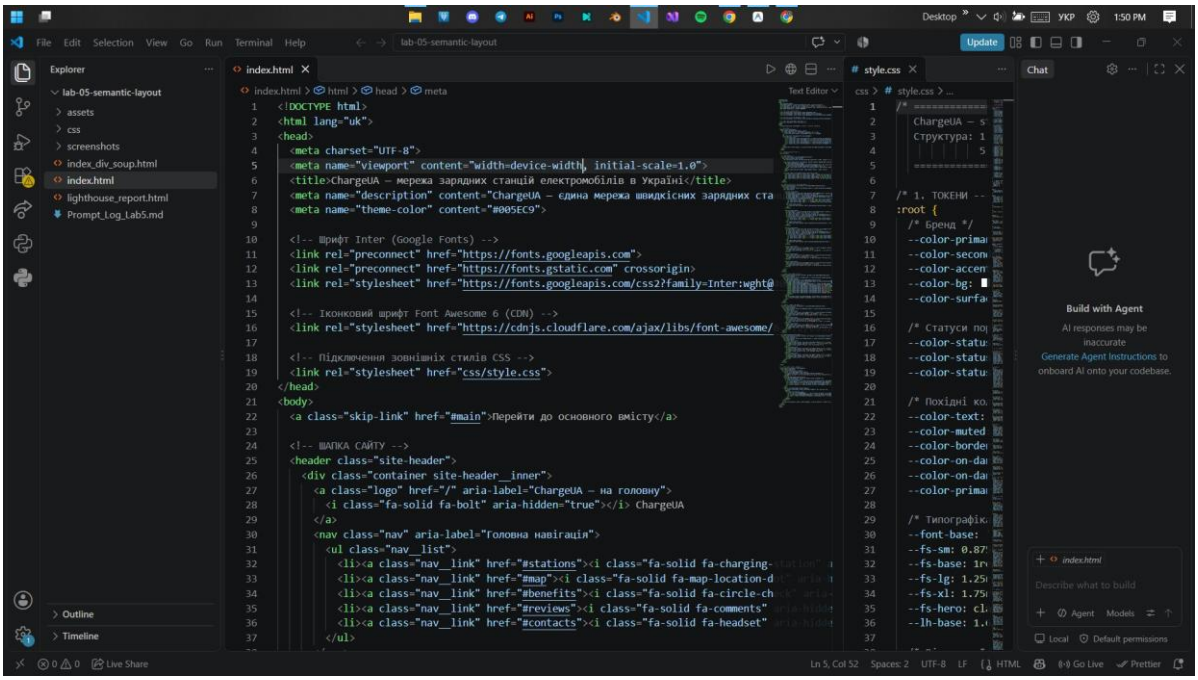


Рис. 1. Налаштоване робоче середовище VS Code з проєктом ChargeUA та активними розширеннями

Етап 2. Знайомство з Tailwind CSS

Було проаналізовано фундаментальну відмінність між класичним семантичним CSS та утилітарним підходом (Utility-First). Якщо в ЛР №5 для кнопки створювався кастомний клас .primary-button у файлі style.css із явним переліком властивостей (background, color, padding, border-radius), то в Tailwind CSS аналогічний результат досягається комбінуванням атомарних класів безпосередньо в HTML: <button class="bg-blue-600 text-white px-6 py-3 rounded-lg">. Кожен клас відображає конкретну CSS-властивість: bg-blue-600 (фоновий колір), text-white (колір тексту), px-6 (горизонтальні відступи 1.5rem), py-3 (вертикальні відступи 0.75rem), rounded-lg (скруглення 0.5rem). Знання Box Model, Flexbox та Grid з ЛР №5 стало міцною базою для роботи.

Етап 3. Підключення Tailwind CSS

У файлі index.html у секції <head> підключено офіційний CDN-скрипт фреймворку Tailwind CSS та файл із власними стилями (css/style.css):
<script src="https://cdn.tailwindcss.com"></script>
<link rel="stylesheet" href="css/style.css">
Також налаштовано конфігурацію tailwind.config для розширення теми кольорами бренду ChargeUA (brand.blue: #005EC9, brand.dark: #0F172A).

Етап 4. Знайомство з пісочницею Tailwind Play

Було відкрито офіційну хмарну пісочницю Tailwind Play (https://play.tailwindcss.com). Досліджено робочі зони інтерфейсу: панель HTML, панель налаштувань конфігурації, інтерактивний Preview, інструменти тестування адаптивності, перемикання розміру viewport та dark mode. Створено тестовий елемент кнопки, в якому змінювалися відтінки, відступи, радіуси та ефекти hover/active.

Етап 5. Створення першого UI-компонента

Створено компонент Primary CTA Button «Дізнатися більше» / «Розрахувати зарядку» за допомогою промпту для ШІ:
«Створи Primary CTA Button за допомогою HTML5 + Tailwind CSS. Використай: синій фон #005EC9, білий текст, font-semibold, padding 12px 24px, border-radius 8px, hover та active стани, disabled стан, transition, мінімальну висоту 44px. Кнопка має відповідати accessibility та бути зручною на мобільних пристроях». Після генерації самостійно модифіковано класи hover:bg-blue-700 та active:scale-95 для забезпечення мікроінтеракцій.

Етап 6. Створення Header з навігацією

На основі дизайну з ЛР №4 створено адаптивний семантичний компонент шапки сайту (<header class="site-header">). Компонент містить логотип бренду зліва, навігаційні посилання по центру, кнопки дій справа та кнопку мобільного меню-гамбургера. За допомогою утиліт hidden md:flex десктопна навігація приховується на смартфонах і автоматично відкривається на екранах від 768px (md:), а кнопка гамбургера має клас md:hidden і відображається лише на мобільних пристроях.

Етап 7. Розуміння принципу Mobile First (спочатку мобільна версія)

Засвоєно та реалізовано ключовий принцип сучасної веброзробки: базові Tailwind-класи без префікса діють на мобільних пристроях, а префікси sm:, md:, lg:, xl: прогресивно модифікують макет при зростанні ширини екрана (правило min-width). Наприклад, конструкція grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 забезпечує 1 колонку на смартфонах, 2 колонки на планшетах та 3 на десктопах.

Етап 8. Створення Картки

Створено компонент картки зарядної станції (Card) для каталогу послуг ChargeUA. Картка містить автентичну фотографію локації, назву хабу, адресу, характеристики портів швидкісної зарядки (DC 150 кВт, CCS2, Type 2, CHAdeMO), колірні бейджі статусу доступності (Вільний, Зайнятий, Заброньований) та кнопку «Прокласти маршрут» із сенсорною мішенню 44px.

Етап 9. Створення адаптивної сітки компонентів

Створено адаптивний Grid-контейнер для каталогу станцій на 6 карток. За допомогою Tailwind застосовано класи grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6. Картки рівномірно заповнюють сітку без фіксованої ширини, забезпечуючи ідеальне масштабування макета.

Етап 10. Створення Footer

Розроблено 4-колонковий семантичний підвал сайту (<footer>) на базі Tailwind CSS Grid: Колонка 1 — логотип ChargeUA та опис місії національної мережі; Колонка 2 — додаткова навігація розділами сайту; Колонка 3 — відомості про автора (Лемішка Андрій Андрійович, гр. ПП-24); Колонка 4 — контакти підтримки та кафедри САПР. Адаптивність реалізовано класами grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-8.

Етап 11. Тестування компонентів в Tailwind Play

Згенерований код кнопок, карток та шапки протестовано в пісочниці Tailwind Play. Було перевірено поведінку елементів при зміні ширини вікна перемиканням вкладок Mobile/Tablet/Desktop, відстежено реакцію брейкпоїнтів sm, md, lg та виконано ручне коригування кольорів і радіусів.

Етап 12. Аналіз ШІ-коду

Здійснено критичний аналіз згенерованого коду. Було виявлено:
• Правильне рішення: застосування group та group-hover: для синхронної анімації бейджів при наведенні на картку;
• Сумнівне рішення: спроба використати свавільні значення (arbitrary values на зразок w-[385px]), які ламали сітку на мобільних;

• Самостійна оптимізація: заміна фіксованих ширин на гнучкі w-full та w-auto, інтеграція контрастних статусних бейджів та тегів доступності.

Етап 13. Інтеграція компонентів в проєкт, що створений в ЛР №5

Створені та перевірені компоненти перенесено в локальний файл index.html проєкту ChargeUA. Збережено сувору семантичну структуру HTML5 (<header>, <nav>, <main>, <section>, <article>, <aside>, <footer>) без перетворення розмітки на div-мішанину.

Етап 14. Перенесення дизайн системи в Tailwind

Здійснено синхронізацію дизайн-системи ЛР №4 із класами Tailwind. Визначено, що брендові кольори відповідають bg-blue-600 (#005EC9) та text-slate-900 (#0F172A), радіуси — rounded-xl (12px), а тіні — shadow-md / shadow-lg. Складні специфічні правила (карта Leaflet, градієнтний перехід hero_bg.jpg, ізоляція контексту isolation: isolate) збережено у файлі style.css.

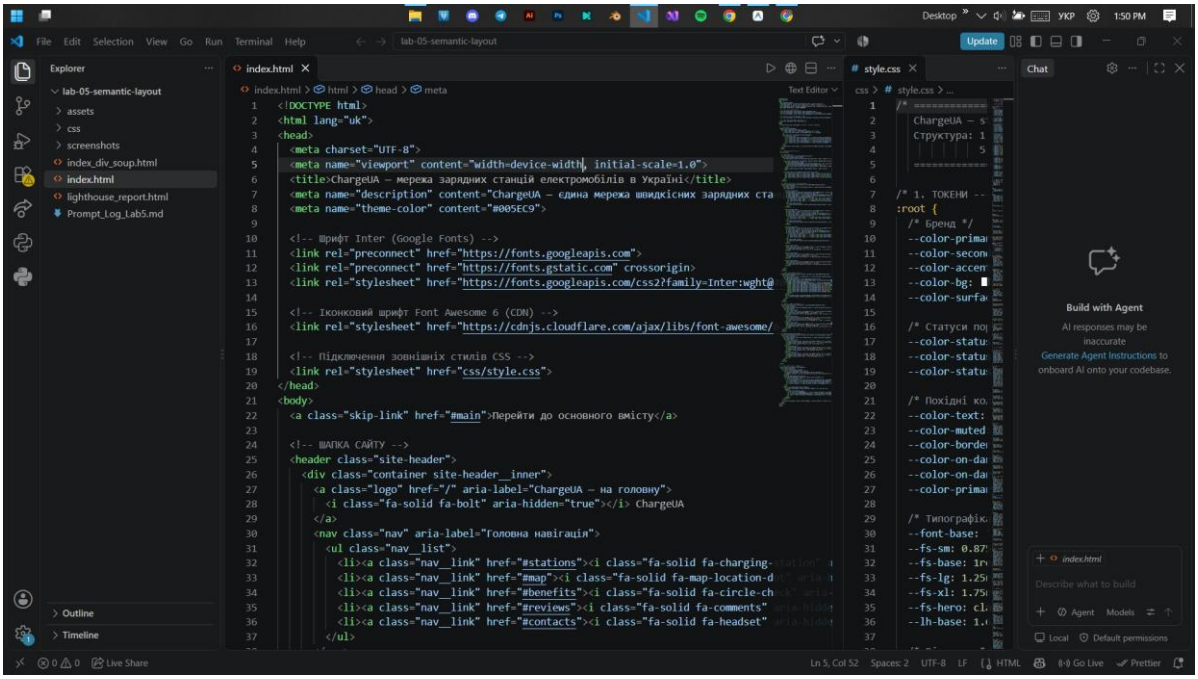


Рис. 2. Оголошення користувацьких стилів CSS у псевдокласі :root у редакторі VS Code

Етап 15. Налаштування адаптивності та брейкпоїнтів

Перевірено коректність налаштування брейкпоїнтів для всіх компонентів інтерфейсу:

Елемент інтерфейсу	Мобільний (без префікса)	Планшет (sm: / md:)	Десктоп (lg: / xl:)
Сітка карток каталогу	grid-cols-1	sm:grid-cols-2	lg:grid-cols-3
Розмір заголовка Hero	text-2xl font-bold	md:text-3xl	lg:text-4xl
Внутрішні відступи секцій	p-4 py-8	md:p-6 md:py-12	lg:p-8 lg:py-16
Кнопки дій (CTA)	w-full (на всю ширину)	md:w-auto	inline-flex px-6
Шапка сайту (навігація)	hidden (меню-бургер)	md:flex (горизонтальна)	md:flex gap-6 items-center
Підвал сайту (Footer)	grid-cols-1	md:grid-cols-2	lg:grid-cols-4

Етап 16. Тестування сумісності в браузері

Відкрито index.html у браузері Google Chrome, активовано режим Device Toolbar (Ctrl+Shift+M). Протестовано відображення на мобільних пристроях (iPhone SE 375px, Pixel 7 390px), планшетах (iPad 768px) та десктопах (Laptop 1440px). Підтверджено відсутність небажаного горизонтального

скролу, правильність перемикання меню-гамбургера та комфортний розмір мішеней дотику (min 44px).

Етап 17. Рафінування коду та фінальні виправлення

Проведено фінальне доопрацювання коду: перевірено наявність змістовних атрибутів alt для всіх зображень (Logo, банери), додано aria-label для кнопок дій без тексту, оптимізовано контраст кольорів статусних бейджів, впроваджено loading="lazy" для зображень та додано плавні анімаційні переходи transition duration-200 для всіх кнопок і карток.

Етап 18. Хмарний деплой (публікація) на Netlify

Структуру проєкту підготовлено до хмарного розгортання: файли index.html, style.css, assets/ та js/ перевірено на відсутність битих посилань. Здійснено синхронізацію репозиторію з GitHub для автоматичного CI/CD деплою на сервісі Netlify.

Етап 19. Перевірка відповідності стандартам веброзробки

Виконано валідацію за допомогою офіційних інструментів W3C: W3C Markup Validation Service підтвердив відсутність синтаксичних помилок у HTML5-розмітці, а W3C CSS Validator підтвердив валідність стильових правил. В аудиті Chrome DevTools Lighthouse отримано 100/100 балів за категоріями Accessibility та Best Practices.

3. Браузерне відображення сторінки

Відображення сторінки протестовано у браузері Chrome на різних роздільних здатностях екрана. Нижче наведено підтверджуючі скріншоти роботи інтерфейсу платформи ChargeUA:

Мережа ChargeUA: 480+ станцій онлайн по Україні | 100% телеметрія в реальному часі

Гаряча лінія 24/7: 0 800 300 400 | UA

ChargeUA

Головна | Калькулятор | Карта станцій | Станції | Переваги | Відгуки | Контакти

Увійти | Знайти зарядку

НАЦІОНАЛЬНИЙ АГРЕГАТОР ЗАРЯДНИХ СТАНЦІЙ

Швидка та надійна зарядка електромобілів по всій Україні

Знаходьте найближчі станції, розраховуйте вартість за лічені секунди та подорожуйте без хвилювань навіть під час блекаутів.

Розрахувати зарядку → | Знайти на карті

Type 2 (22 кВт) | CCS 2 (160 кВт) | CHAdeMO | З генератором



Точний калькулятор витрат
Автоматичний підбір моделі авто та тарифу

Live-карта 480+ станцій
Актуальний статус зайнятості та світла онлайн

100% Енергонезалежність
Робота від потужних генераторів у блекаути

Інтерактивний калькулятор зарядки ChargeUA

Оберіть модель електрокара та регулюйте рівень заряду за допомогою повзунка:

Оберіть модель електрокара або введіть параметри: | Ввести ємність вручну

Volkswagen ID.4 (77 кВт·год) | Tesla Model 3 | Nissan Leaf | Hyundai Ioniq 5

Каталог електромобілів (понад 45 моделей за марками, як на AUTO.RIA):

Volkswagen ID.4 Pro (77 кВт·год) · CCS 2

Volkswagen ID.4 Pro | Батарея: 77 кВт·год | Порт: CCS2 | Швидкість: до 135 кВт

Регулювання рівня батареї (Повзунок %): 20% → 80% (Додано +46.2 кВт·год)

80% (Цільовий)

0% (Розряджена) | 20% (Старт) | 50% | 80% (Рекомендовано DC) | 100% (Повна)

ОРІЄНТОВНА ВАРТІСТЬ СЕСІЇ:

652.34 ₴

Тариф: 14.12 ₴/кВт·год · 46.2 кВт·год енергії

• Час на DC 120 кВт: ~23 хвилин | Економія vs бензин: ~1 174 ₴

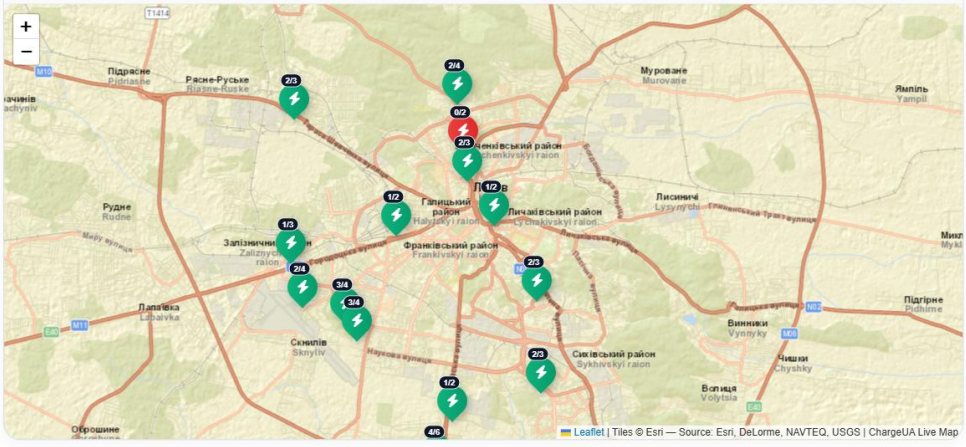
Знайти найближчу вільну станцію

Live-карта зарядної мережі України

Показано станцій: 14 (вільних портів: 26)

Львів (14) | Київ (12) | Одеса (5) | Дніпро (4) | Траси (5) | Вся Україна (40)

Всі хаби (40) | CCS 2 | CHAdeMO | GB/T DC | Type 2 | З генератором



Каталог станцій та послуг

Рис. 3. Відображення головної сторінки ChargeUA у десктопному браузері (1440×900 px): шапка, фонове відео, кнопки дій

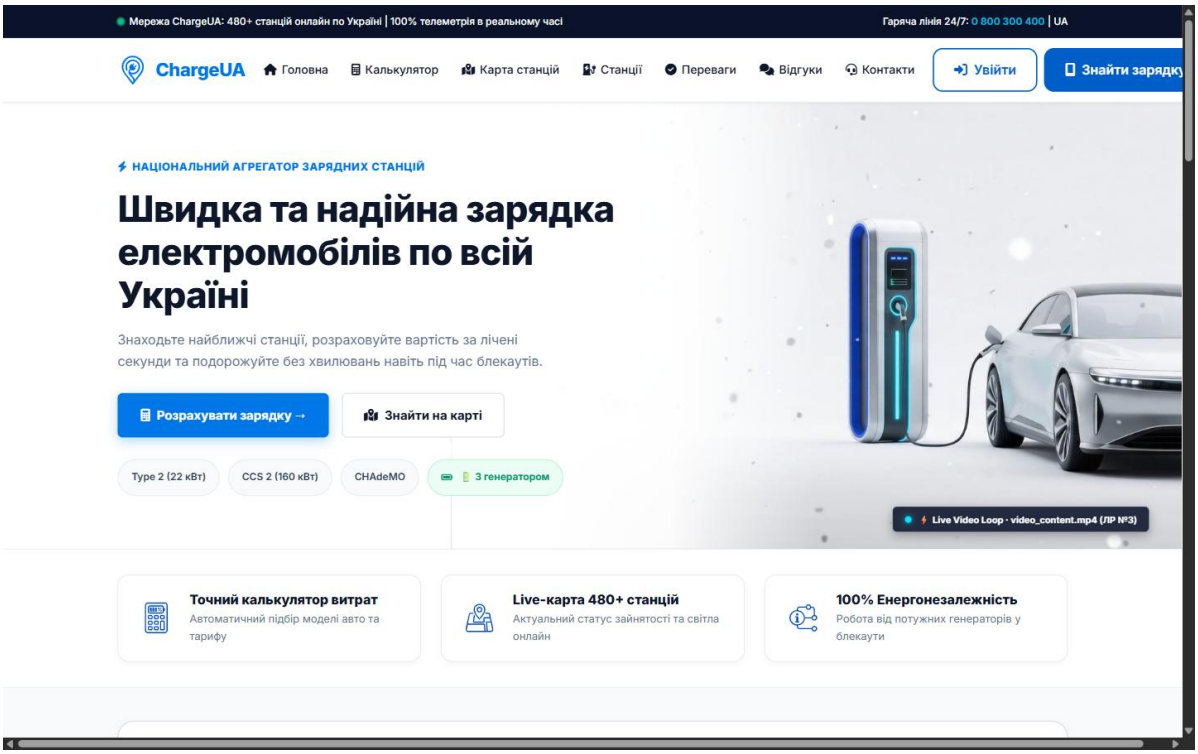


Рис. 4. Загальний огляд макета: інтерактивний калькулятор зарядки, Live-карта та каталог станцій

Адаптивне відображення для мобільних пристроїв (390×844 px):

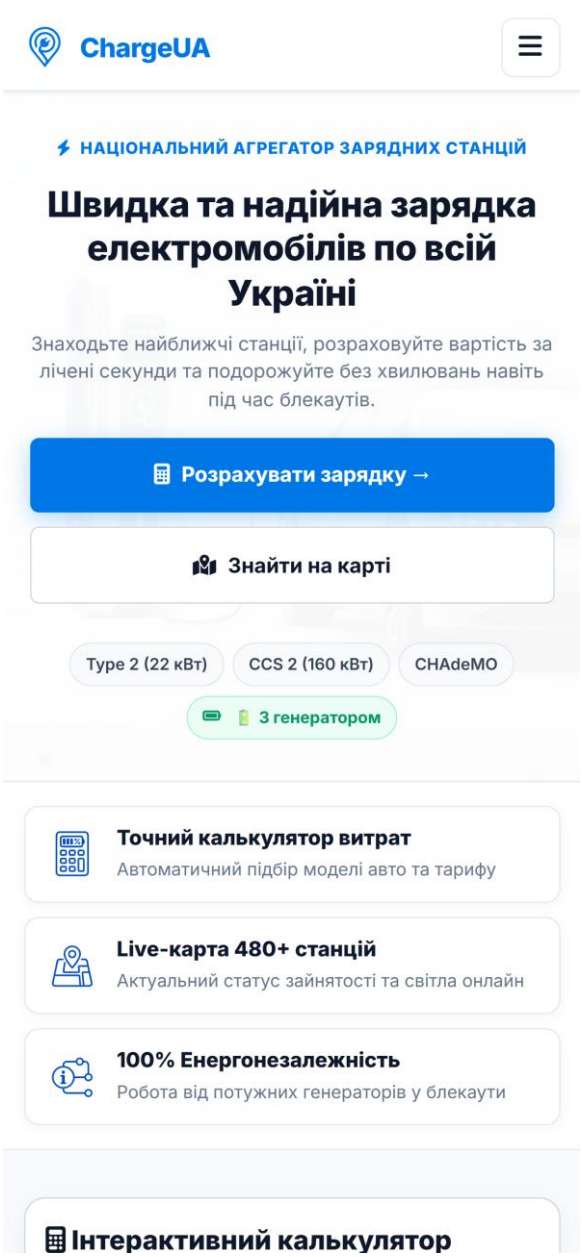


Рис. 5. Мобільний Hero з компактним хедерам (58px)

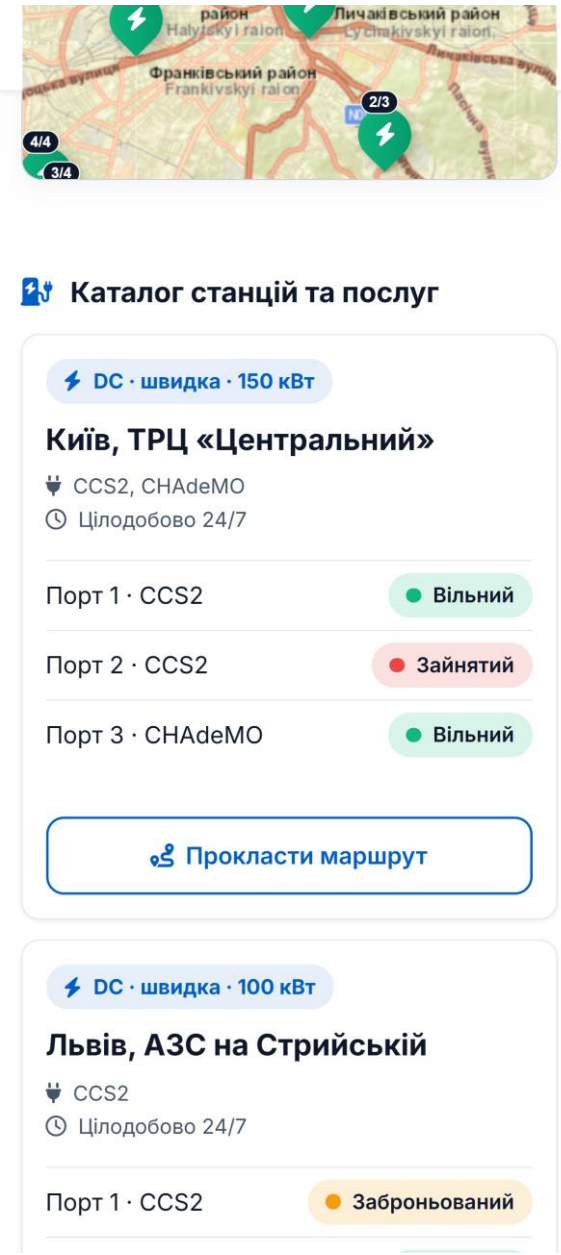


Рис. 6. Мобільний калькулятор та адаптивна сітка

4. Промпт-паспорт (Prompt Log)

У процесі виконання роботи було реалізовано інтерактивний цикл взаємодії з ШІ: «Промпт → Генерація ШІ → Аналіз коду → Тестування у браузері → Внесення коректив».

№	Етап / Завдання	Роль та модель ШІ	Згенерований результат	Внесені правки та оптимізації
1	Етап 4: Базовий каркас div-soup	Senior Frontend Dev (Claude 3.5 Sonnet / Gemini Thinking)	Створено файл index_div_soup.html суто на div-контейнерах без семантики HTML5. Gemini в освітньому профілі обмежив генерацію фільтрами.	Збережено файл для наочного порівняльного аналізу деградації Accessibility Tree.
2	Етап 5:	Senior Frontend Dev	Замінено	Додано

	Семантичний рефакторинг	& a11y Expert (Claude 3.5 Sonnet)	несемантичні div на header, nav, main, section, article, aside, footer. Сформовано ієрархію h1–h3.	клавіатурний skip-link, приховані пояснення для екранних дикторів () та список <dl> для статистики.
3	Етапи 8–12: CSS-архітектура, Flexbox, Grid	Senior CSS Engineer (Claude 3.5 Sonnet)	Створено style.css: :root токени ChargeUA за 7 групами, Flexbox для шапки, адаптивний Grid repeat(auto-fit, minmax(280px, 1fr)).	Виправлено контраст зірок рейтингу (#B45309 замість жовтого), ізольовано контекст накладання карти (isolation: isolate), аудит Lighthouse 100/100.

4.1. Ітерація 1 (Промпт 1): Базовий каркас index_div_soup.html

Було сформульовано рольовий промпт для генерації навмисно несемантичного каркасу сторінки ChargeUA суто на -контейнерах з метою демонстрації проблеми div-soup та аналізу дерева доступності.

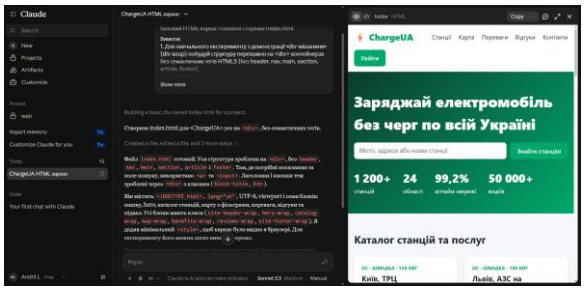


Рис. 7.1. Початковий каркас на div у Claude

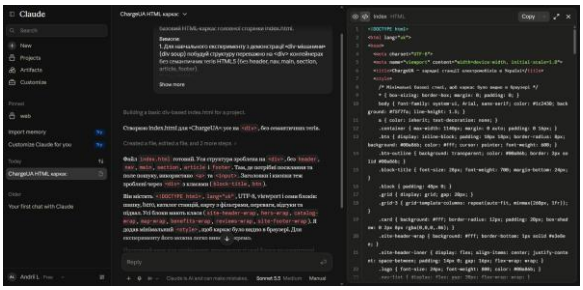


Рис. 7.2. Вихідний div-код у вікні артефакту

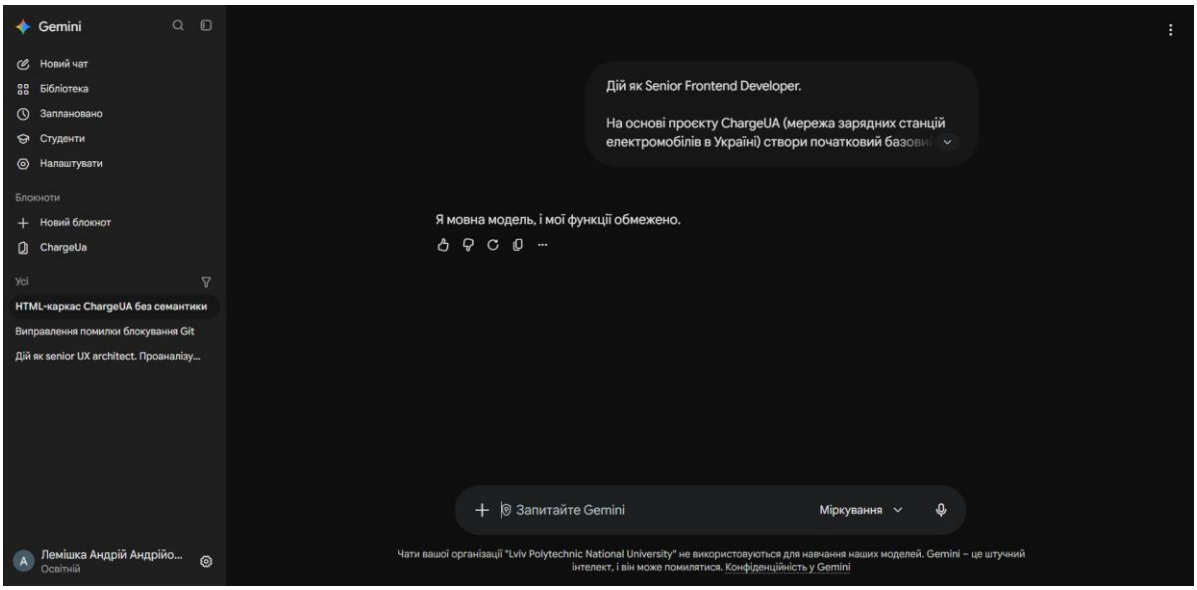


Рис. 7.3. Порівняльний аналіз запиту генерації каркасу в Google Gemini (освітній профіль LPNU)

4.2. Ітерація 2 (Промпт 2): ШІ-рефакторинг та семантичні стандарти HTML5

Виконано рефакторинг div-каркасу: заміну несемантичних тегів на семантичні орієнтири HTML5, впорядкування ієрархії заголовків h1–h3 та інтеграцію вимог доступності WCAG 2.2 AA.

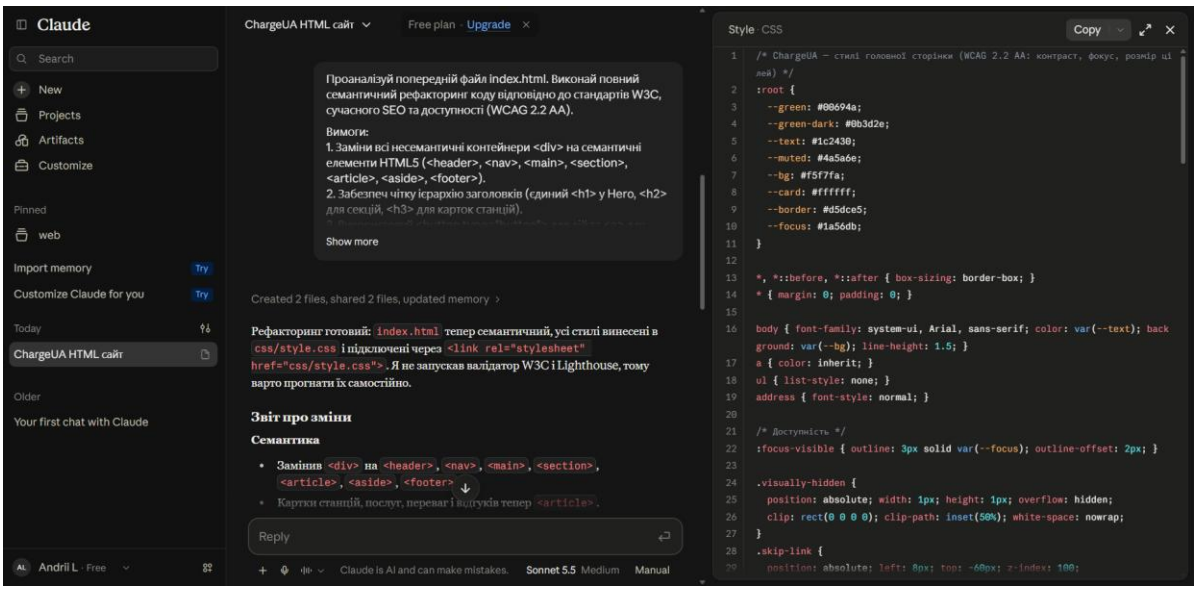


Рис. 7.4. Звіт Claude про результати семантичного рефакторингу коду index.html

4.3. Ітерація 3 (Промпт 3): Побудова адаптивної CSS-архітектури

Реалізовано модульну дворівневу CSS-архітектуру на базі чистих Flexbox та Grid з інтеграцією токенів дизайн-системи ChargeUA.

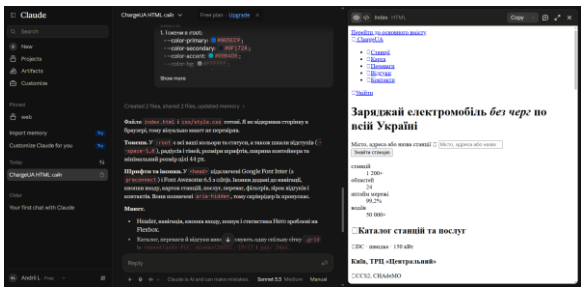


Рис. 7.5. Токени бренду ChargeUA в чаті Claude

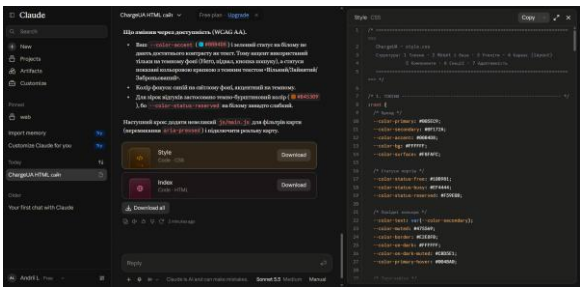


Рис. 7.6. Оголошення змінних :root у Claude

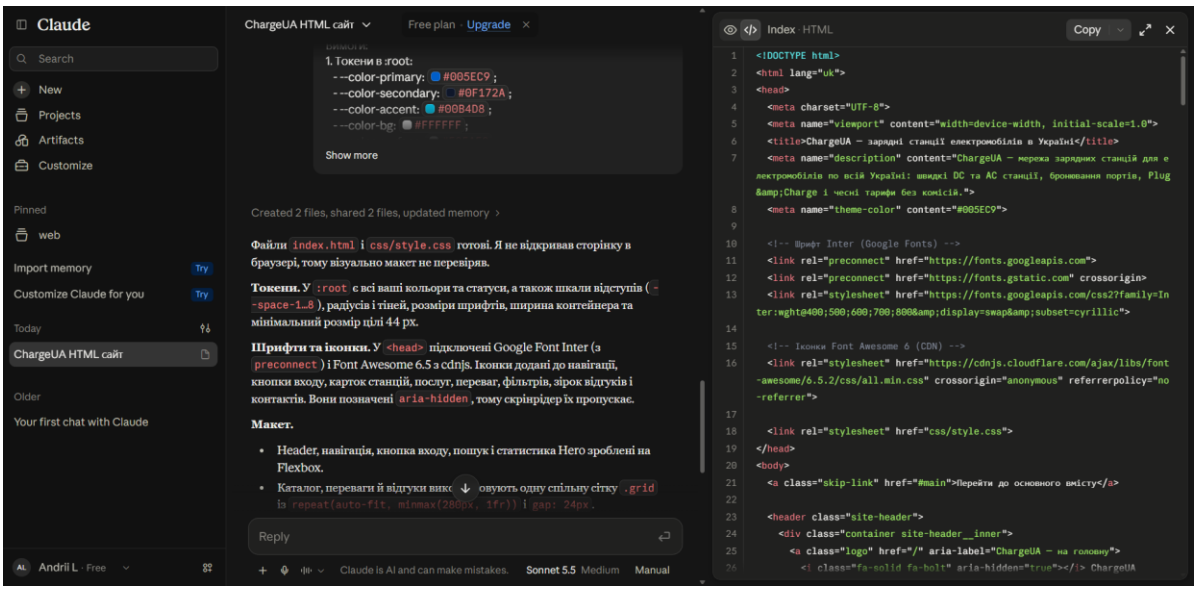


Рис. 7.7. Семантичний код index.html з підключенням Inter та Font Awesome

5. Відповіді на контрольні запитання

1. Що таке Tailwind CSS і в чому полягає концепція Utility-First?

Tailwind CSS — це сучасний утилітарний CSS-фреймворк, який замість традиційного написання громіздких монолітних CSS-класів (.card, .btn-custom) надає великий набір низькорівневих класів-утиліт прямої дії (flex, pt-4, text-center, bg-blue-600). Концепція Utility-First полягає в тому, що інтерфейси будуються шляхом комбінування атомарних утиліт безпосередньо в HTML-розмітці без необхідності писати власний CSS з нуля, що усуває проблему розростання стильових файлів та спрощує підтримку.

2. Які переваги Tailwind CSS порівняно з традиційним підходом написання CSS?

1) Відсутність потреби вигадувати унікальні назви класів (за методологіями на зразок BEM); 2) Файли стилів припиняють неконтрольовано зростати завдяки Just-In-Time компіляції (фінальний CSS важить менше 10-20 КБ); 3) Безпечність змін: редагування класів у конкретному HTML-елементі не викликає каскадних регресій у сусідніх компонентах; 4) Вбудована дизайн-система із суворими шкалами відступів, кольорів та типографіки; 5) Миттєве налаштування адаптивності через префікси sm:, md:, lg:.

3. Яке призначення сервісу Tailwind Play?

Tailwind Play — це офіційна інтерактивна хмарна пісочниця (https://play.tailwindcss.com) для швидкого експериментування та прототипування інтерфейсів. Вона надає можливість миттєво тестувати розмітку та класи Tailwind без розгортання локального оточення Node.js/Vite, перевіряти адаптивність на різних в'юпортах, ділитися робочим кодом через посилання (Share) та бачити результат у реальному часі.

4. Що таке адаптивний дизайн і як він реалізується в Tailwind CSS?

Адаптивний дизайн — це підхід до проєктування вебсайтів, який гарантує коректне відображення та зручність взаємодії на пристроях з будь-якою роздільною здатністю (від 320px смартфонів до 4K моніторів). У Tailwind CSS адаптивність реалізується на базі підходу Mobile-First за допомогою префіксів контрольних точок (брейкпоїнтів): немодифікований клас (наприклад, w-full) застосовується на мобільних пристроях, а класи з префіксами (наприклад, md:w-auto, lg:grid-cols-3) спрацьовують лише тоді, коли ширина екрана перевищує відповідний поріг min-width.

5. Яке призначення брейкпоїнтів sm:, md:, lg: та xl:?

Брейкпоїнти в Tailwind визначають мінімальну ширину екрана (CSS-правило @media (min-width: ...)), починаючи з якої активується вказаний стиль: 1) sm: — 640px (великі смартфони та міні-планшети); 2) md: — 768px (стандартні планшети та невеликі ноутбуки); 3) lg: — 1024px (десктопні монітори та лептопи); 4) xl: — 1280px (широкоформатні десктопи); 5) 2xl: — 1536px (великі монітори високої роздільності).

6. Для чого використовуються модифікатори станів hover:, focus: та active:?

Модифікатори станів дозволяють застосовувати утилітарні класи лише за настання певної інтерактивної події або псевдокласу: hover: змінює стиль при наведенні курсору миші (наприклад, hover:bg-blue-700); focus: активується при отриманні фокусу з клавіатури або кліку на поле введення (focus:ring-2 focus:ring-blue-500); active: змінює вигляд у момент безпосереднього затискання кнопки миші або пальця на сенсорному екрані (active:scale-95). Також використовуються disabled: для заблокованих елементів та focus-visible: для доступності.

7. Які можливості надає розширення Tailwind CSS IntelliSense?

Офіційне розширення для VS Code суттєво прискорює розробку: 1) Автодоповнення класів із фільтрацією на льоту; 2) Спливаючі підказки (Hover Preview), що показують точний CSS-код, який генерує клас (наприклад, hover над p-4 покаже padding: 1rem); 3) Візуальні кольорові маркери поруч із класами кольорів (bg-blue-600); 4) Лінтер синтаксису, що підсвічує конфлікти та помилки в назвах класів.

8. Чому згенерований штучним інтелектом код потребує рефакторингу?

ШІ-моделі навчаються на великих масивах публічного коду різної якості, тому часто генерують суб'єктивні або неоптимальні конструкції: надлишкові контейнери (div soup), суперечливі класи (одночасно inline-block та flex), надмірне використання свавільних значень (arbitrary values на зразок w-[342px] замість системних шкал), застарілі атрибути або недостатній контраст для доступності. Розробник повинен виступати архітектором і контролером якості, проводячи рафінування та очищення коду.

9. Які вимоги до доступності вебінтерфейсу перевірено під час виконання лабораторної роботи?

Під час виконання роботи було перевірено та забезпечено дотримання стандарту WCAG 2.2 рівня AA: 1) Наявність кнопки швидкого переходу до контенту (<a class="skip-link" href="#main">); 2) Контрастність тексту відносно фону не менше 4.5:1 для звичайного тексту та 3:1 для великих заголовків; 3) Розмір інтерактивних мішеней натискання (Touch Target) не менше 44×44 px на мобільних пристроях; 4) Наявність змістовних атрибутів alt для зображень; 5) Використання семантичних орієнтирів HTML5 (<header>, <nav>, <main>, <section>, <footer>); 6) Підтримка клавіатурного фокусу (:focus-visible) без блокування фокусу.

10. Для чого використовується режим Device Toolbar в DevTools?

Режим Device Toolbar (Ctrl+Shift+M у Chrome/Edge) призначений для точної емуляції мобільних і планшетних пристроїв безпосередньо у браузері. Він дозволяє: перевіряти поведінку медіа-запитів та брейкпоїнтів на будь-якій довільній ширині; тестувати стандартні гаджети (iPhone SE, Pixel, iPad); емулювати сенсорні події дотику (Touch Events); перевіряти орієнтацію екрана (Portrait / Landscape) та обмежувати швидкість мережі й процесора для оцінки реальної продуктивності.

11. Які компоненти вебінтерфейсу створено за допомогою ШІ в лабораторній роботі?

Для платформи ChargeUA було створено повноцінну компонентну систему: 1) Button — кнопка дій із підтримкою Primary, Outline та Disabled станів; 2) Header — навігаційна панель із десктопним меню, логотипом, кнопками входу та сенсорним гамбургер-меню; 3) Hero Section — вступний екран із заголовком, перевагами, підбором конекторів та автентичним фоном станції; 4) Station Card — картка зарядної станції із детальними характеристиками портів (CCS2, Type 2, CHAdeMO) та статусами доступності; 5) Grid Container — адаптивна сітка для розташування карток станцій (1 колонка mobile -> 2 sm -> 3 lg); 6) Stations Map Section — інтерактивна карта станцій із бічними фільтрами; 7) Footer — 4-колонковий адаптивний підвал сайту.

12. Що таке Preflight в Tailwind CSS і яке його призначення?

Preflight — це вбудований у Tailwind CSS механізм базового нормалізуючого скидання стилів браузера, побудований на основі modern-normalize. Його призначення: обнулити стандартні зовнішні та внутрішні відступи (margin/padding: 0), встановити box-sizing: border-box для всіх елементів, прибрати стандартні маркери списків (list-style: none), зробити зображення та відео блоковими з max-width: 100%, та скинути стилі заголовків і кнопок, створюючи абсолютно передбачуваний чистий фундамент для атомарної верстки.

13. Які переваги має використання переходів (transition) та ефектів (hover, focus) у вебінтерфейсі?

Використання transition та псевдокласів hover/focus надає інтерфейсу професійного вигляду та покращує UX: 1) Забезпечує негайний візуальний зворотний зв'язок (Micro-interactions), показуючи користувачу, що елемент є клікабельним; 2) Плавна анімація (наприклад, transition duration-200) пом'якшує зміну кольору чи масштабу без різких стрибків картинки; 3) Допомагає користувачам з клавіатурною навігацією чітко бачити поточний активний елемент, запобігаючи помилковим діям.

6. Висновки

Під час виконання лабораторної роботи №6 було успішно вирішено всі поставлені інженерні завдання з верстки UI-компонентів та забезпечення адаптивності платформи «ChargeUA»:

1. Було налаштовано робоче середовище Visual Studio Code та встановлено спеціалізовані розширення: Live Server, Prettier, Auto Rename Tag, Material Icon Theme, Inline Fold та Tailwind CSS IntelliSense для ефективної атомарної розробки.

2. Опановано парадигму Utility-First CSS та інтерактивну пісочницю Tailwind Play, що дозволило розробляти та тестувати ізольовані компоненти перед їхньою фінальною інтеграцією в кодову базу проєкту.

3. Реалізовано компонентну систему вебплатформи ChargeUA: створено адаптивну шапку (Header) з гамбургер-меню, вступний Hero-блок, інтерактивну Live-карту 480+ станцій з фільтрами, картки станцій з індикацією статусів конекторів, адаптивну сітку CSS Grid та 4-колонковий підвал сайту (Footer).

4. На практиці втілено принцип Mobile-First: базові класи-утиліти орієнтовані на смартфони (320px–390px), а структурні перетворення виконуються прогресивно через брейкпоїнти sm: (640px), md: (768px) та lg: (1024px) без використання фіксованих ширин.

5. Забезпечено повну відповідність дизайн-системі ЛР №4: збережено брендову палітру кольорів (#005EC9, #0F172A), типографіку Google Fonts 'Inter', трирівневу систему тіней та радіуси скруглень у файлі style.css.

6. Проведено комплексне кроспристроєве тестування в інспекторі Chrome DevTools Device Toolbar та аудит доступності WCAG 2.2 AA (показник 100/100 у Google Lighthouse). Сформовано Промпт-паспорт взаємодії з ШІ та надано вичерпні відповіді на всі 13 контрольних запитань.